



Faculdade de Informática e Administração Paulista

Alan Junio Araujo de Souza - RM 574112

Arthur Vettorazzo de Souza – RM 569445

Brayan Barbosa Dos Santos - RM 573682

Giovanne Gomes Petenuci - RM 574091

Gustavo Zibini Belizario - RM 561376

Luiz Otávio Brito Freixo - RM 569977

Desafio FIAP + GoodWe EV Challenge 2026

Sprint 3 – Prototipagem Funcional e Integração

1CCPZ (CIÊNCIAS DA COMPUTAÇÃO - NOTURNO)

São Paulo / SP

2026



Alan Junio Araujo de Souza - RM 574112

Arthur Vettorazzo de Souza – RM 569445

Brayan Barbosa Dos Santos - RM 573682

Giovanne Gomes Petenuci - RM 574091

Gustavo Zibini Belizario - RM 561376

Luiz Otávio Brito Freixo - RM 569977

Desafio FIAP + GoodWe EV Challenge 2026

Sprint 3 – Prototipagem Funcional e Integração

Trabalho apresentado à Faculdade de Informática e Administração Paulista – FIAP como parte das exigências acadêmicas referentes à Sprint 3 do Desafio FIAP + GoodWe EV Challenge 2026: prototipagem funcional e integração do sistema ChargeGrid Intelligence — NexusCharge.

São Paulo / SP

2026

Sumário

1. O problema e o que construímos.....	4
2. Esquema de integração	5
2.1 Diagrama de blocos.....	5
2.2 Fluxograma da decisão de potência.....	7
3. Justificativa técnica.....	8
3.1 Por que Modbus TCP e não OCPP	8
3.2 Por que SQLite e o módulo sqlite3 da biblioteca padrão	8
3.3 Por que as sessões ativas ficam em memória e só as pagas vão ao banco	9
3.4 Como o carregamento é liberado — e por que o pagamento, não o encerramento, libera o conector	9
4. Resultados e dados funcionais.....	10
4.1 Orquestração de potência num posto de 33 kW	10
4.2 Tarifação nos três eixos.....	10
4.3 Comunicação Modbus	11
4.4 Histórico acumulado	11
4.5 Testes	12
5. O sistema funcionando	12
6. Fundamentos aplicados: pensamento computacional e gestão de energia	15
6.1 Modularização: separar para não repetir o erro do arquivo que faz tudo	15
6.2 Importação de módulos próprios e ponto de entrada.....	16
6.3 O loop de interação e a validação que falha cedo	16
6.4 Dicionários e tuplas como estrutura de configuração	17
6.5 JSON como linguagem entre sistemas	17
6.6 Memória versus disco: eficiência computacional como eficiência energética	18
6.7 Caminhos com pathlib e o arquivo que o sistema acha sozinho	18
6.8 Interceptar o erro em vez de deixar quebrar	18
6.9 Matrizes e classificação por faixa.....	19
6.10 Onde as duas frentes se encontram	19
7. Sustentabilidade, eficiência energética e a experiência do usuário.....	19
8. Como executar	21

ChargeGrid Intelligence — NexusCharge

Prototipagem funcional e integração — Sprint 3 · EV Challenge 2026 · FIAP + GoodWe · Time 6

Nome	RM
Giovanne Gomes Petenuci	574091
Arthur Vettorazzo de Souza	569445
Gustavo Zibini Belizario	561376
Alan Junio Araujo de Souza	574112
Brayan Barbosa Dos Santos	573682
Luiz Otávio Brito Freixo	569977

Repositório: <https://github.com/FIAP-Time6-GoodWe-2026/ChargeGridIntelligence-NexusCharge-Sprint3>

1. O problema e o que construímos

Um eletroposto comercial com cinco carregadores GoodWe GW11K-HCA-20 tem um problema de aritmética que parece banal e não é: cinco vezes 11 kW são 55 kW, e o disjuntor do prédio aguenta 33 kW. Ou o estabelecimento paga uma obra de entrada de energia, ou alguém precisa decidir, a cada carro que chega, quanta potência cada um recebe.

O ChargeGrid Intelligence é esse alguém. É um sistema web em Python que orquestra a potência entre os conectores, calcula a tarifa de cada sessão, conversa com o carregador pelo protocolo industrial Modbus TCP, cobra o motorista e guarda o histórico. Ele roda com python app.py e depende de duas bibliotecas: flask e pytest.

O que muda para quem usa: o motorista abre o mapa, escolhe um conector e começa a carregar sem saber que o sistema acabou de reduzir a potência de outros três carros para caber no disjuntor. O dono do posto vê no painel quanto está sendo consumido e quanto já faturou. Ninguém precisa ir até o quadro de energia.

2. Esquema de integração

2.1 Diagrama de blocos

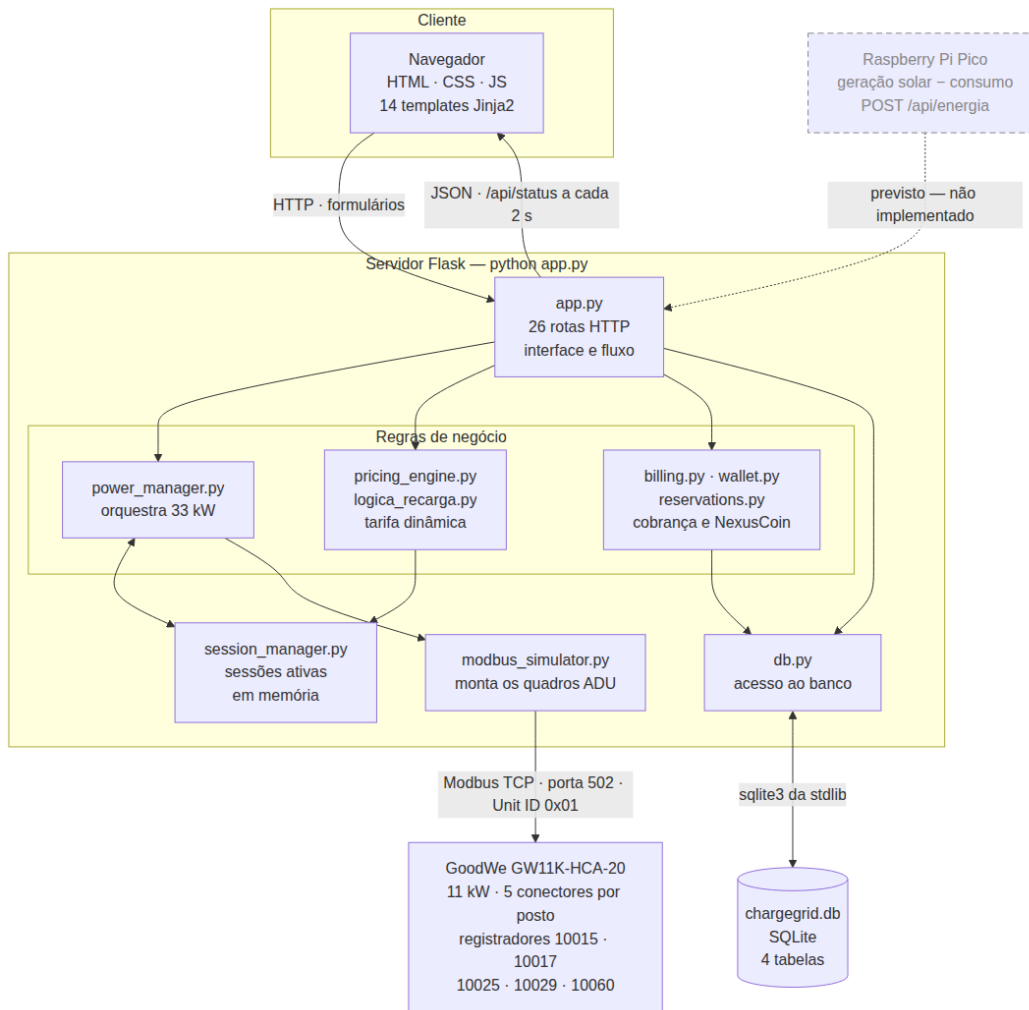


Figura 1 — Diagrama de blocos: como os módulos, o banco e o carregador se ligam.

O caminho de uma requisição é sempre o mesmo: o navegador manda um formulário HTTP, o app.py recebe na rota correspondente, chama a regra de negócio que resolve aquilo, e a regra fala com quem guarda o dado. Nenhuma rota abre o banco diretamente e nenhum módulo de regra sabe o que é uma requisição HTTP.

São 16 módulos Python e 14 templates. Os quatro que fazem o trabalho pesado:

Módulo	Linhas	Responsabilidade
app.py	1889	26 rotas HTTP — recebe, valida e devolve
power_manager.py	526	decide quanta potência cada sessão recebe
session_manager.py	520	guarda as sessões ativas em memória
modbus_simulator.py	481	monta os quadros Modbus para o carregador

O bloco tracejado é o Raspberry Pi Pico, responsável por calcular geração solar – consumo e devolver o orçamento de potência disponível. O contrato da API já está definido — um POST em `/api/energia` com dois números em JSON — mas a integração não foi implementada nesta Sprint: hoje o limite de 33 kW é um parâmetro do PowerManager, não um valor que vem do Pico.

2.2 Fluxograma da decisão de potência

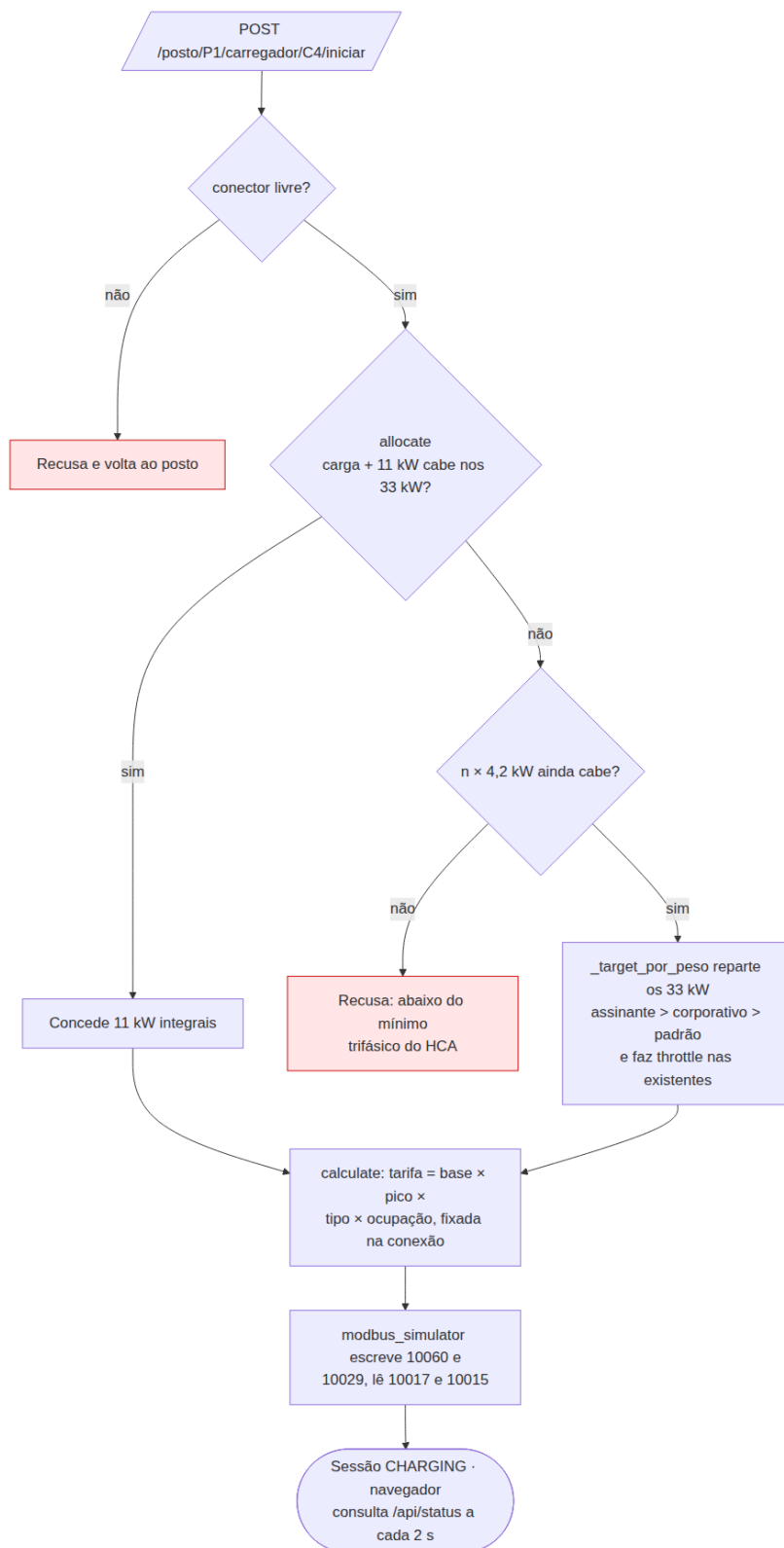


Figura 2 — Fluxograma da decisão de potência, do clique no conector até a sessão carregando.

A parte interessante é o losango do meio. Quando o carro que chega não cabe, o sistema não recusa nem estoura o limite: ele redistribui. `_target_por_peso` reparte os 33 kW entre todas as sessões — as que já estavam e a nova — usando um peso por tipo de usuário, de modo que assinante fica acima de corporativo, que fica acima de padrão.

Há um piso físico. O HCA G2 trifásico não opera abaixo de 4,2 kW, e um carregador que recebe menos que isso desliga. Por isso existe o segundo losango: se $n \times 4,2$ kW já não cabe no limite, a conexão é recusada com uma mensagem explicando o motivo, em vez de aceitar o carro e derrubar o posto.

3. Justificativa técnica

3.1 Por que Modbus TCP e não OCPP

A escolha óbvia para eletropostos seria o OCPP, que é o padrão de comunicação entre carregador e plataforma de gestão. Em mentoria com a GoodWe confirmamos que a linha HCA G2 não suporta OCPP. O que o GW11K-HCA-20 expõe é Modbus TCP na porta 502, com registradores documentados. Então é Modbus que o sistema fala.

Os registradores que o sistema movimenta:

Registrador	Direção	Uso
10015	leitura	potência entregue no momento
10016	leitura	energia acumulada da sessão
10017	leitura	status do carregador
10025	escrita	controle dinâmico de carga, ligado ao redistribuir
10026	escrita	corrente máxima do disjuntor da instalação
10029	escrita	potência máxima permitida ao conector
10060	escrita	habilita e desabilita o carregamento

O `modbus_simulator.py` monta o quadro ADU completo — cabeçalho MBAP mais PDU — e registra cada um num log consultável. Não é uma representação aproximada: o byte a byte é o mesmo que sairia na rede.

3.2 Por que SQLite e o módulo `sqlite3` da biblioteca padrão

Um posto de recarga precisa lembrar de saldos, extratos e sessões pagas depois de reiniciar. Um banco relacional resolve isso, e considerar PostgreSQL foi o primeiro impulso

— mas PostgreSQL exige um servidor rodando, e este é um protótipo que precisa ser executável com dois cliques.

O sqlite3 já vem com o Python. Não instala nada, não sobe serviço, e o banco é um arquivo ao lado do código. Optamos também por não usar ORM: as consultas são SQL direto, e são poucas.

A decisão que mais deu certo foi abrir uma conexão nova por operação, em vez de manter uma aberta. O servidor de desenvolvimento do Flask atende cada requisição numa thread diferente, e o sqlite3 recusa conexões compartilhadas entre threads. Abrir e fechar por chamada custa microssegundos e eliminou a classe inteira de erro.

3.3 Por que as sessões ativas ficam em memória e só as pagas vão ao banco

Uma sessão em andamento muda de estado a cada dois segundos — energia acumulada, custo parcial, potência após um throttle. Gravar isso no disco a cada ciclo seria escrita constante para um dado que só interessa enquanto o carro está plugado.

Então há duas vidas. Enquanto carrega, a sessão vive numa lista dentro do `session_manager.py`. Quando o motorista paga, ela é gravada na tabela `sessoes` e passa a existir independente do processo.

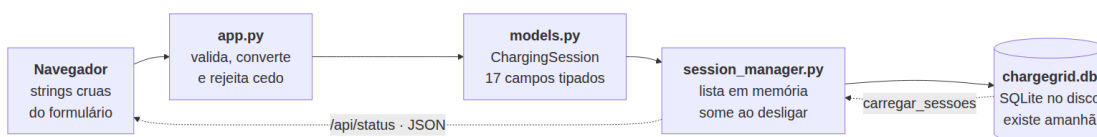


Figura 3 — O caminho de uma sessão, do formulário do navegador até o disco, e a volta.

Isso teve uma consequência visível que confundiu o time durante o desenvolvimento: o relatório consolidado mostrava 9 sessões enquanto o painel do operador mostrava 180. Os dois estavam certos — o relatório lê a memória, o painel lê o banco. Corrigimos os rótulos das telas, que agora dizem "sessões nesta execução" e "no histórico".

3.4 Como o carregamento é liberado — e por que o pagamento, não o encerramento, libera o conector

A primeira versão liberava o conector assim que o motorista encerrava a recarga. Testando, percebemos o buraco: era só encerrar e fechar a aba para sair sem pagar, com o conector já livre para o próximo.

Hoje encerrar interrompe a energia e leva à tela de pagamento, mas o conector continua ocupado. Só a confirmação do pagamento libera a vaga e dispara o rebalanceamento que devolve potência a quem estava em throttle. Em termos de registrador Modbus, "liberar" significa duas escritas em sequência: 10060 recebe o valor que desabilita o carregamento, e só depois, quando a vaga é efetivamente desocupada, o conector volta a aparecer como disponível para o próximo motorista.

4. Resultados e dados funcionais

Todos os números desta seção saíram de executar o sistema, não de estimativa.

4.1 Orquestração de potência num posto de 33 kW

Conectamos cinco carros de 11 kW no mesmo posto, um de cada vez:

Conector	Potência concedida	Potência de cada sessão ativa	Carga total	Ocupação
C1	11,0 kW	11,0	11,0 kW	33,3 %
C2	11,0 kW	11,0 · 11,0	22,0 kW	66,7 %
C3	11,0 kW	11,0 · 11,0 · 11,0	33,0 kW	100,0 %
C4	8,25 kW	8,25 × 4	33,0 kW	100,0 %
C5	6,6 kW	6,6 × 5	33,0 kW	100,0 %

A carga total nunca passa de 33,0 kW. Do quarto carro em diante o sistema reduz todo mundo em vez de recusar: quatro carros a 8,25 kW, cinco a 6,6 kW. Nenhum cai abaixo dos 4,2 kW do piso trifásico — nenhum veículo é interrompido totalmente, apenas recebe menos potência.

Com usuários de tipos diferentes o peso aparece nos números. Num posto cheio com cinco sessões, medimos 7,47 kW para o assinante, 6,84 kW para o corporativo e 6,22 kW para cada um dos três padrão — somando os mesmos 33 kW.

4.2 Tarifação nos três eixos

A tarifa é calculada uma vez, no momento da conexão, e não muda durante a sessão. Partindo da base de R\$ 1,20/kWh:

Hora	Tipo de usuário	Posto vazio	Posto lotado
14h (fora de pico)	Padrão	R\$ 1,20	R\$ 1,56
14h (fora de pico)	Assinante	R\$ 1,02	R\$ 1,326
14h (fora de pico)	Corporativo	R\$ 1,08	R\$ 1,404
19h (pico)	Padrão	R\$ 1,80	R\$ 2,34
19h (pico)	Assinante	R\$ 1,53	R\$ 1,989
19h (pico)	Corporativo	R\$ 1,62	R\$ 2,106

Os três multiplicadores são independentes: horário de pico (18h–22h59) soma 50 %, assinante desconta 15 %, corporativo desconta 10 %, e posto lotado soma 30 %. O pior caso — padrão, no pico, com o posto cheio — é R\$ 2,34/kWh, quase o dobro da base. Há ainda uma cobrança mínima de R\$ 2,00 por sessão, para que uma parada de três minutos não gere uma fatura de centavos.

4.3 Comunicação Modbus

O ciclo de vida das cinco sessões acima gerou 31 quadros Modbus. O primeiro deles, habilitando o carregamento do conector P1-C1:

```
TX      |    FC06    Write    |    Reg    10060    |    Data:    ['0x0002']
HEX: 00 01 00 00 00 06 01 06 27 4C 00 02
```

Lendo o quadro: 00 01 é o identificador da transação, 00 00 o protocolo, 00 06 o tamanho, 01 o endereço do escravo, 06 a função "escrever um registrador", 27 4C o endereço 10060 em hexadecimal e 00 02 o valor.

4.4 Histórico acumulado

Medida	Valor
Sessões pagas no banco	180
Receita faturada	R\$ 4.205,76
Energia entregue	2.878,23 kWh
Duração média	102,5 min
Período	27/07/2026 a 09/09/2026

Esses dados saem em CSV por `/admin/export.csv` e alimentam análises estatísticas complementares.

4.5 Testes

A suíte tem 190 testes cobrindo as regras de negócio: tarifação nos três eixos, redistribuição de potência, piso trifásico, taxa mínima, reserva com sinal, saldo insuficiente, os algoritmos de busca e ordenação, e a separação entre memória e histórico. Roda com `pytest -q`, e também por dentro do navegador, em `/testes`.

5. O sistema funcionando

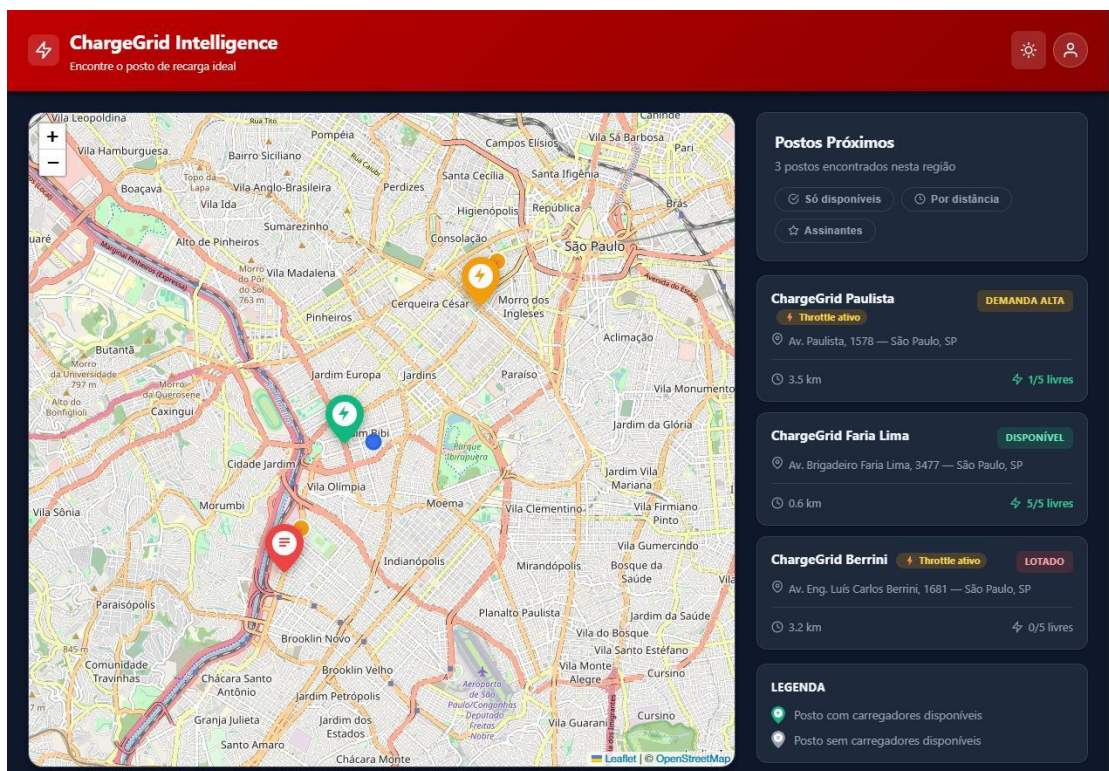


Figura 4 — Mapa dos três postos com o estado de cada um em tempo real.

O mapa marca cada posto com seu estado real: Paulista e Berrini com throttle ativo, Faria Lima livre. Os selos de demanda e a contagem de conectores livres vêm do mesmo cálculo que o painel do operador usa. É aqui que a experiência do usuário se conecta diretamente à automação de back-end: a cor do pino no mapa não é decorativa, ela é a leitura, em tempo real, do mesmo `occupancy_ratio()` que o `power_manager.py` usa para decidir a tarifa e o throttle.

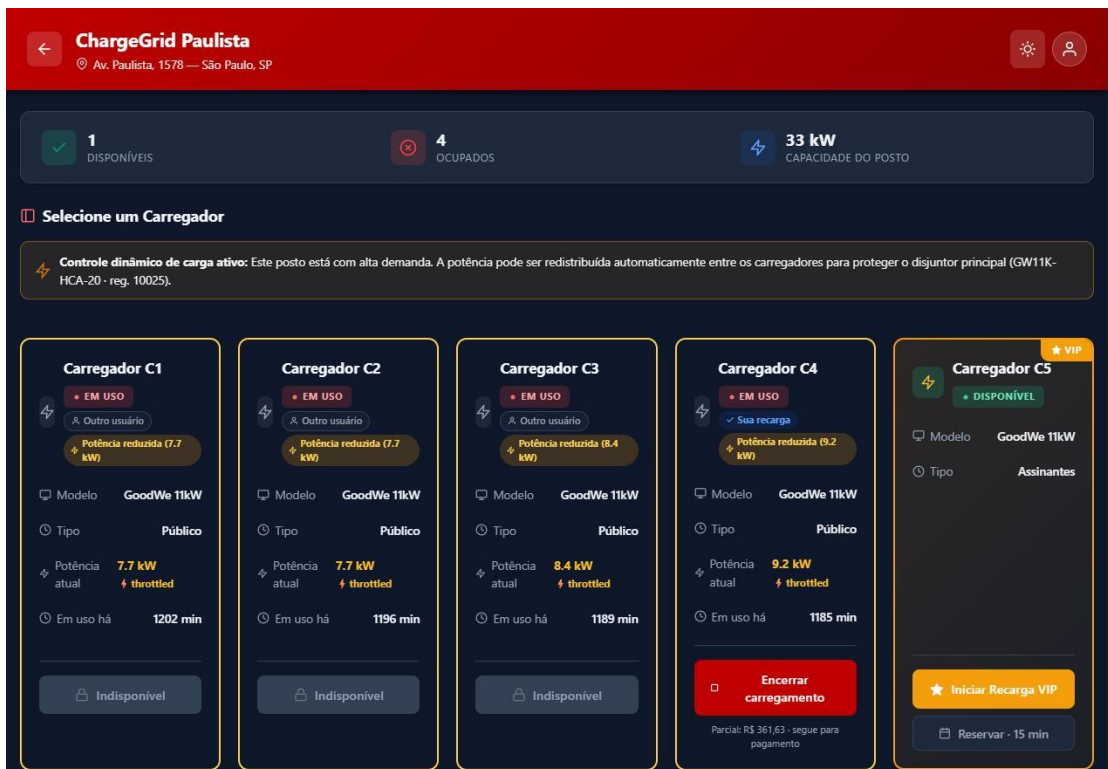


Figura 5 — Posto Paulista: quatro conectores em uso, todos com potência reduzida pela orquestração.

Dentro do posto, o efeito da orquestração fica explícito em cada card: quatro conectores em uso, todos marcados "potência reduzida", com os valores individuais — 7,7 kW, 7,7 kW, 8,4 kW e 9,2 kW. O aviso no topo explica ao motorista por que a potência dele não é 11 kW — nenhum veículo aparece como desconectado ou interrompido, porque nenhum foi.

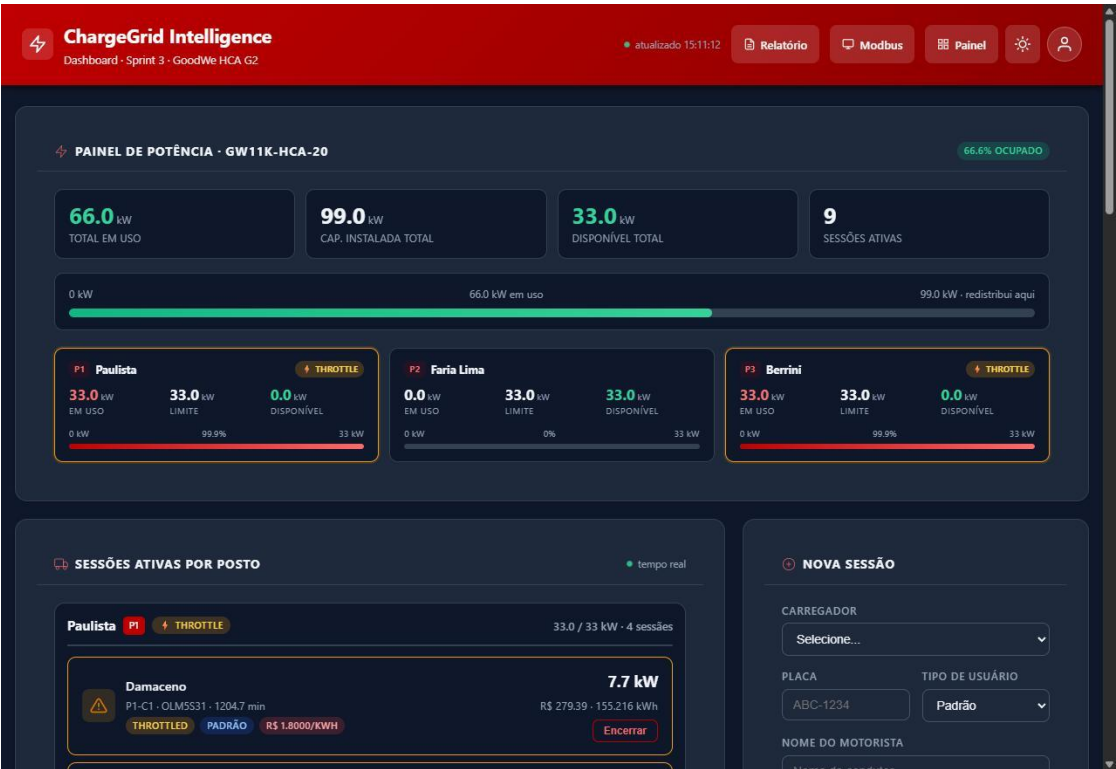


Figura 6 — Painel de potência por posto e sessões ativas, atualizado a cada dois segundos.

O painel mostra os três postos lado a lado com a barra de ocupação, e a lista de sessões ativas com potência, tarifa fixada e custo parcial de cada uma. Atualiza sozinho a cada dois segundos, consultando a mesma rota /api/status que o mapa e a tela do posto usam — é o único ponto de verdade sobre o estado da rede.

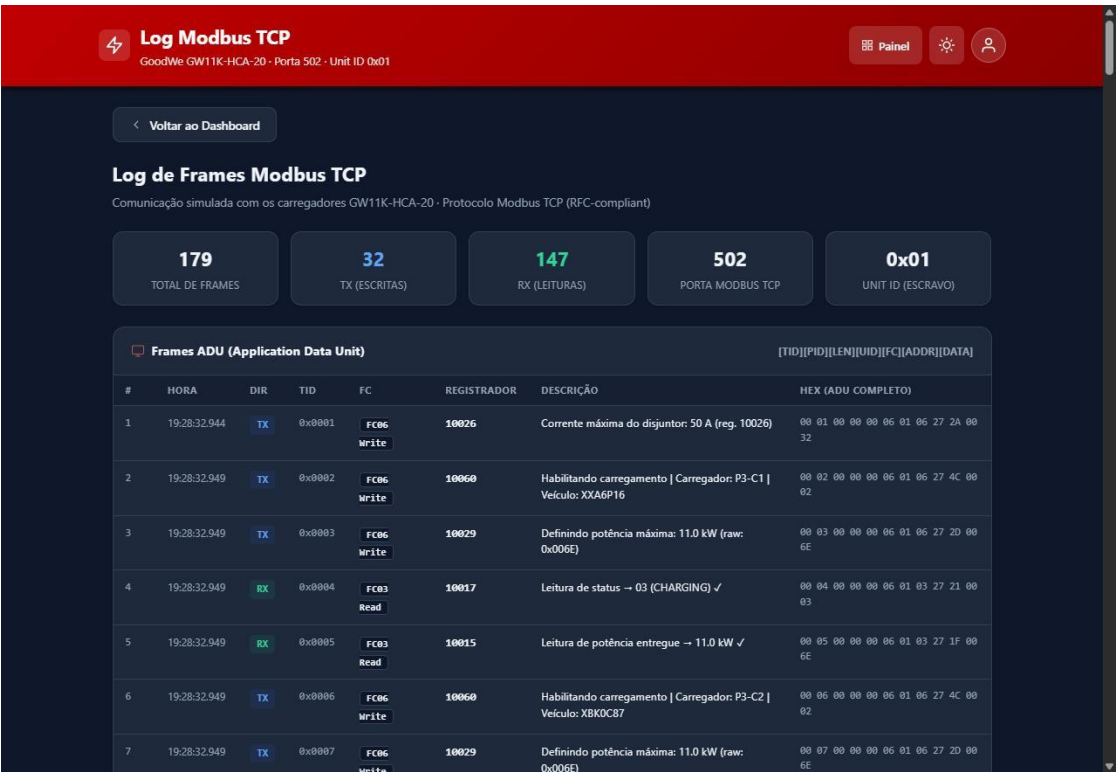


Figura 7 — Log de quadros Modbus TCP com o ADU completo em hexadecimal.

O log de Modbus é onde a camada de automação fica visível: cada quadro com direção, identificador de transação, código de função, registrador, descrição legível e o ADU completo em hexadecimal.

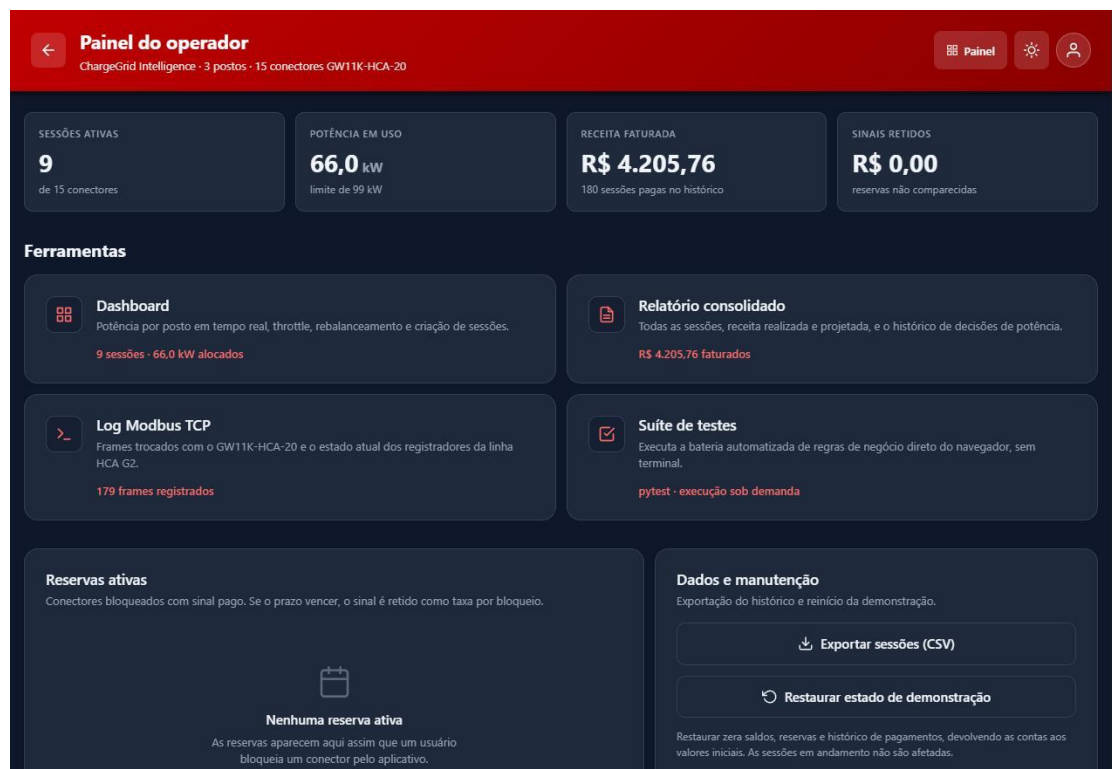


Figura 8 — Painel do operador com os indicadores do negócio.

E o painel do operador reúne os indicadores do negócio — sessões ativas, potência em uso, receita faturada e sinais retidos de reservas não cumpridas.

6. Fundamentos aplicados: pensamento computacional e gestão de energia

O ChargeGrid Intelligence nasceu como exercício de automação com Python, mas por natureza é também um sistema de gestão de energia: a mesma lógica que organiza módulos de código é a que organiza quilowatts num disjuntor. Esta seção percorre os fundamentos que sustentam o projeto nos dois eixos — o de engenharia de software e o de soluções energéticas — mostrando onde eles se encontram.

6.1 Modularização: separar para não repetir o erro do arquivo que faz tudo

Um sistema pequeno tende a virar um único arquivo que lê entrada, decide e grava dado, tudo junto — o padrão que qualquer projeto de automação aprende a evitar. O ChargeGrid separa três papéis: interface e fluxo, modelagem de regras, e acesso a dados.

Papel	Módulo(s)
Interface e fluxo	app.py — 26 rotas, nenhuma linha de SQL
Modelagem de dados e regras	models.py, logica_recarga.py, pricing_engine.py, power_manager.py
Acesso e persistência	db.py — único módulo que importa sqlite3
Dados persistidos	chargegrid.db

O db.py segue a convenção de isolar o acesso ao dado atrás de uma função privada — `_conn()` (db.py:104) — e o resto do sistema usa `execute`, `query_one` e `query_all`. Nenhuma rota sabe que existe um cursor. Essa separação é também a que qualquer gestão de energia precisa: quem decide quanto liberar para cada carga não deveria precisar saber como o dado chega até o disjuntor, e vice-versa.

Vale registrar uma parte que não está resolvida: com 1889 linhas e 26 rotas, o app.py já é grande demais para um arquivo só. A separação correta seria dividir por assunto — recarga, carteira, painel — usando Blueprints do Flask. Isso ficou para a Sprint 4.

6.2 Importação de módulos próprios e ponto de entrada

Há uma diferença entre importar uma biblioteca externa e importar código próprio, e o ChargeGrid tem 36 arestas de import entre os próprios módulos. A guarda `if __name__ == "__main__"` aparece nos dois pontos de entrada: app.py:1831 sobe o servidor Flask, menu.py:712 abre o menu de terminal. Ela existe para que importar o módulo não dispare o programa — e no projeto ela é usada de verdade: a suíte de testes importa app.py para testar as rotas, e o servidor não sobe.

6.3 O loop de interação e a validação que falha cedo

O menu.py segue o padrão clássico de interação por terminal: um `while True` (menu.py:685) exibindo o menu, uma cadeia de opções, e a saída por uma opção dedicada.

A validação segue a ideia de rejeitar antes de processar. `ler_numero` (menu.py:121) cobre as três formas de erro:

```
try:
    valor = int(bruto) if inteiro else float(bruto.replace(",", "."))
except ValueError:
    restam = TENTATIVAS_MAXIMAS - tentativa
```

```

tipo = "número inteiro" if inteiro else "número"
aviso(f"{bruto}' não é um {tipo}."
+ (f'Restam {restam} tentativa(s)." if restam
else "Tentativas esgotadas."))
continue

```

Texto onde se espera número vira aviso, não exceção. Valor fora da faixa é recusado com a faixa na mensagem. Enter vazio cancela. A função nunca levanta exceção — devolve None e quem chamou decide.

6.4 Dicionários e tuplas como estrutura de configuração

Dicionários são a estrutura de configuração do sistema inteiro. O mapa de registradores Modbus (modbus_simulator.py:79) é um dicionário de nome para endereço; os três postos são um dicionário indexado pelo identificador; os critérios de ordenação (algoritmos.py:231) mapeiam a opção digitada para o par nome/função.

O caso mais claro é o `_chargers` do `session_manager.py`: 15 conectores com identificador fixo, consultados por nome o tempo todo — a vantagem do dicionário é que a chave pode ser o que fizer sentido, aqui "P1-C4" em vez de uma posição numérica que ninguém lembra.

Tuplas aparecem onde o retorno tem duas partes que andam juntas e não devem ser alteradas: as funções de busca devolvem (posição, comparações).

6.5 JSON como linguagem entre sistemas

JSON transcende o Python: é como se conversa com qualquer API na web, e é exatamente o papel dele aqui. A rota `/api/status` (app.py:1033) devolve o estado inteiro da rede em JSON, e o JavaScript de cada tela busca esse endereço a cada dois segundos:

```

{
  "potencia_em_uso": 65.95,
  "potencia_limite": 99.0,
  "postos": {
    "P1": {"em_uso_kw": 32.98, "limite_kw": 33.0, "ocupacao_pct": 99.9, "sessoes": 4}
  },
  "sessoes_ativas": [
    {"charger_id": "P3-C5", "allocated_kw": 7.47, "status": "THROTTLED",
     "tariff_kwh": 1.989, "energy_kwh": 0.523, "user_type": "A"}
  ]
}

```

```
]
}
```

É o mesmo formato que o Pico usará para publicar geração e consumo quando a integração for feita — o ponto de encontro natural entre o sistema de software e o sistema de energia.

6.6 Memória versus disco: eficiência computacional como eficiência energética

A diferença entre memória RAM, onde o dado sobrevive só enquanto o programa está aberto, e a persistência, que faz a informação existir amanhã, foi uma decisão consciente de projeto, descrita na seção 3.3: sessão em andamento na memória, sessão paga no disco.

A volta do disco para a memória é a `carregar_sesoes()` (`db.py:217`), que lê a tabela e reconstrói os objetos `ChargingSession`. O ganho não é só de organização de código: evitar escrita constante em disco a cada dois segundos é o mesmo princípio de eficiência que evitar desperdício de energia — fazer o mínimo de trabalho necessário, no momento certo, em vez de repetir uma operação cara sem necessidade.

6.7 Caminhos com `pathlib` e o arquivo que o sistema acha sozinho

`Path(__file__).resolve().parent` é a forma de o sistema achar seus próprios arquivos independentemente de onde o terminal foi aberto. O `db.py:43` usa a mesma ideia:

```
DB_PATH: pathlib.Path = pathlib.Path(__file__).with_name("chargegrid.db")
```

Com isso `python app.py` funciona de qualquer diretório de trabalho, e o `iniciar.bat` pode ser clicado de qualquer lugar.

6.8 Interceptar o erro em vez de deixar quebrar

Se o banco estiver corrompido ou ausente, o histórico volta vazio e o sistema continua de pé (`db.py:242`):

```
except sqlite3.Error as erro:
    logger.warning("Histórico indisponível (%s) — carregando vazio.", erro)
    return []
```

É uma decisão de design: o sistema abre com a coleção vazia em vez de mostrar um `traceback` — o mesmo raciocínio de tolerância a falhas que uma rede elétrica real precisa ter quando um sensor para de responder.

6.9 Matrizes e classificação por faixa

O `power_manager.py` calcula a ocupação percentual do posto e classifica a decisão em três níveis — "ok", "warning" e "danger" — no campo `severity` do `AllocationResult`. O template usa esse campo para escolher a cor do card, em vez de procurar palavras dentro da mensagem — o mesmo tipo de raciocínio usado para classificar o estado de um sistema por faixas de percentual, seja de sucesso de um serviço ou de ocupação de um posto de recarga.

O paralelo com a matriz também existe: a rede é uma tabela de 3 postos × 5 conectores, percorrida para somar potência por posto e por rede.

6.10 Onde as duas frentes se encontram

O que torna o ChargeGrid um bom terreno comum entre automação e soluções energéticas é que a mesma decisão aparece dos dois lados. Do lado do software, `power_manager.py` é uma função de alocação de recursos sob restrição — um problema clássico de algoritmos. Do lado da energia, essa função é literalmente um gestor de demanda: ela pratica load shedding proporcional (reduzir carga em vez de cortar totalmente), prioriza consumidores por classe (assinante, corporativo, padrão — equivalente a contratos de demanda diferenciados), e aplica tarifação dinâmica por horário de pico, o mesmo princípio por trás da tarifa branca residencial ou de programas de resposta à demanda em redes reais.

A arquitetura modular descrita nas seções anteriores é o que torna essa lógica de energia testável, auditável e substituível — hoje o limite de 33 kW é um número fixo, e a próxima etapa natural é ele vir de uma medição real de geração solar, o que aproxima ainda mais o sistema de um caso de uso de gestão energética completo.

7. Sustentabilidade, eficiência energética e a experiência do usuário

Três pontos merecem destaque especial: como o carregamento é liberado, como a demanda é redistribuída sem interromper totalmente os veículos, e como o sistema se conecta à experiência do usuário. As seções técnicas deste documento já respondem cada um deles, e este parágrafo resume onde:

Como o carregamento é liberado: apenas após a confirmação do pagamento, não no encerramento da recarga (seção 3.4) — a vaga só é anunciada como livre quando não há mais cobrança pendente sobre ela.

Como a demanda é redistribuída sem interromper totalmente os veículos: o `power_manager.py` reduz a potência de todas as sessões ativas proporcionalmente ao tipo de usuário, e nunca desliga um carregador — o piso de 4,2 kW garante que a redução pare antes do ponto em que o HCA G2 se desligaria (seções 2.2 e 4.1).

Como o sistema se conecta à experiência do usuário: o mapa muda de cor em tempo real com o mesmo cálculo de ocupação que orquestra a potência, o conector VIP prioriza assinantes, e o checkout com cashback em NexusCoin fecha o ciclo comercial sem que o motorista precise entender nada do que acontece no back-end (seção 5).

O ganho ambiental do sistema não está em gerar energia, e sim em não desperdiçar infraestrutura. Sem orquestração, atender cinco carros a 11 kW exigiria 55 kW de entrada — obra de padrão, transformador maior, mais cobre. Com orquestração, os mesmos cinco carros carregam dentro de 33 kW já instalados. É menos obra e menos pico de demanda sobre a rede.

Cada peça técnica do sistema também converte diretamente em ganho de sustentabilidade:

Redistribuição de potência evita sobredimensionar a instalação elétrica do posto — menos cobre e menos obra de entrada de energia para atender a mesma frota de veículos.

Tarifa de demanda e de horário de pico (+30 % e +50 %) empurra o consumo para fora do intervalo de maior estresse da rede elétrica (18h–23h) — o mesmo princípio da tarifa branca residencial, aplicado à recarga.

Piso de 4,2 kW no Modbus evita ciclos de liga/desliga no carregador, que degradariam a vida útil do hardware HCA G2 mais rápido do que uma redução suave e contínua de potência.

Sessão ativa em memória, só a paga vai ao SQLite reduz a escrita constante em disco — eficiência computacional do próprio sistema, não só do produto físico.

O passo que falta — e é o que ligaria o sistema à geração renovável de fato — é o orçamento de potência vir de uma medição solar em vez de ser fixo. É a integração com o Pico descrita na seção 2.1.

8. Como executar

Windows: dois cliques em iniciar.bat. Ele procura o Python, instala flask e pytest se faltarem, e abre um menu com três opções — aplicativo web, menu de terminal e testes. A opção 1 sobe o servidor e abre o navegador em <http://localhost:5001>.

Qualquer sistema:

```

pip install flask pytest
python app.py # http://localhost:5001
python menu.py # menu de terminal
pytest -q # 190 testes

```

Contas de demonstração, todas com senha 1234:

Usuário	Perfil	Saldo	Serve para demonstrar
amanda	Assinante	100 NC	recarga, pagamento e cashback
allan	Corporativo	5 NC	saldo insuficiente e recarga da carteira
jose	Padrão	0 NC	carteira zerada
mylon	Operador	10.000 NC	painel, dashboard e log Modbus

O repositório tem o código-fonte, os templates, a suíte de testes, o `seed_historico.py` que gera a base de demonstração e o README com a arquitetura detalhada.